KISVÁRDAI SZC FEHÉRGYARMATI PETŐFI SÁNDOR TECHNIKUM
4900 Fehérgyarmat, Május 14. tér 16.
203043/003

Szoftverfejlesztő és - tesztelő technikus képzés
5 0613 12 03

VIZSGAREMEK

SzakiPark

Az iskola előtti parkolások megkönnyítése
weboldal és mobilalkalmazás segítségével.

Konzulens:
Lakatos Sándor
Berki Balázs

Készítette:
Korpás Miklós

FEHÉRGYARMAT
2026

Tartalom

MySQL adatbázis-kezelő rendszer (XAMPP környezetben) 16
Általános bemutatás 16
Alkalmazás a projektben 17
Adatbázis-tervezési szempontok 17
CRUD műveletek megvalósítása 18
Összegzés 18
Node.js futtatókörnyezet 19
Általános bemutatás 19
Alkalmazás a projektben 20
Express.js keretrendszer alkalmazása 20
Adatbázis-kapcsolat 21
Kliens–szerver kommunikáció 21
Összegzés 21
Frontend technológiák 22
Frontend réteg szerepe 22
A frontend réteg főbb feladatai: 22
Tervezési szempontok a projekt során 24
Összegzés 24
Angular keretrendszer 25
Általános bemutatás 25
Az Angular főbb előnyei 25
Alkalmazás a projektben 26
Komponensalapú felépítés 26
REST API kommunikáció 26
Reactive Forms alkalmazása 26
Összegzés 27
TypeScript 28
Általános bemutatás 28
A TypeScript előnyei 28
Alkalmazás a projektben 29
Összegzés 29
HTML és CSS 30
Általános bemutatás 30
HTML szerepe 30
CSS szerepe 30
A HTML és CSS előnyei 31

Alkalmazás a projektben 31
HTML sablonok alkalmazása 32
CSS alkalmazása 32
Összegzés 32
Figma – tervezés 33
Általános bemutatás 33
A Figma főbb előnyei 33
Alkalmazás a projektben 34
Vizuális tervezés 34
Prototípus-készítés 34
Reszponzív tervezés 34
Összegzés 35
Frontend fejlesztési szempontok 36
Reszponzív dizájn 36
Komponensalapú struktúra 36
Állapotkezelés 37
Felhasználói élmény (UX) 37
Összegzés 37
Tesztelés és együttműködés 38
Tesztelési stratégia áttekintése 38
Backend tesztelés 38
Frontend tesztelés 38
Összegzés 39
Jest – Backend tesztelés 40
Általános bemutatás 40
Alkalmazás a projektben 40
Backend metódusok tesztelése 41
API végpontok ellenőrzése 41
Mock objektumok alkalmazása 41
Összegzés 41
Jasmine és Karma – Frontend tesztelés 42
Általános bemutatás 42
Alkalmazás a projektben 42
Komponensek tesztelése 43
Szolgáltatások tesztelése 43
Felhasználói interakciók validálása 43
Összegzés 43

GitHub – Verziókezelés 44
Általános bemutatás 44
Alkalmazás a projektben 44
Discord – Kommunikáció 46
Általános bemutatás 46
3.5.3 Alkalmazás a projektben 46
Modellezési eszközök 48
UML diagram 48
Általános bemutatás 48
Alkalmazás a projektben 48
ER-modell 49
Általános bemutatás 49
Alkalmazás a projektben 49
Összegzés 49

Bevezetés

A vizsgaremekünk témájának kiválasztását egy mindennapi, valós probléma felismerése indokolta. Az iskolánk előtti parkolóterületen rendszeresen nehézséget okoz szabad parkolóhelyet találni, különösen a reggeli órákban, amikor egyszerre érkeznek diákok, tanárok, az intézmény dolgozói, valamint a szülők. A parkoló telítettsége gyakran időveszteséggel és kényelmetlenséggel jár, ami indokolttá tette egy hatékonyabb, átláthatóbb megoldás kidolgozását.

Projektünk célja egy mindenki számára elérhető, webalapú parkolónyilvántartó rendszer megvalósítása volt, amely az iskola előtti parkolóhelyek aktuális állapotának nyomon követését teszi lehetővé. A rendszer segítségével a felhasználók valós idejű információt kaphatnak arról, hogy mely parkolóhelyek szabadok vagy foglaltak, ezáltal megkönnyítve és felgyorsítva a parkolás folyamatát. A megoldás egyik kiemelt funkciója a parkolóhelyek hosszabb távú bérlésének lehetősége, amely biztosítja a bérlők számára a garantált parkolóhelyet, csökkentve ezzel a zsúfoltságot és a bizonytalanságot.

A projekt megvalósításának első lépéseként kiválasztottuk a megfelelő parkolóterületet, amely az iskolánk előtti parkoló volt. Ezt követően közös ötletelés során meghatároztuk a felhasznált technológiákat, fejlesztői eszközöket és a rendszer alapvető funkcionális

követelményeit. A tervezési fázis során elkészítettük az adatbázis logikai felépítését meghatározó ER-modellt, majd erre alapozva UML-diagramot hoztunk létre, amely a rendszer struktúrájának és az egyes komponensek kapcsolatainak áttekintését szolgálta.

A fejlesztés során nagy hangsúlyt fektettünk a csapatmunkára és a verziókezelésre, ezért a projekt megvalósításához a GitHub verziókezelő rendszert használtuk. Ennek segítségével a csapat minden tagja könnyedén hozzáférhetett a forráskódhoz és a kapcsolódó dokumentációkhoz, valamint biztosított volt a változások nyomon követése és a fejlesztési folyamat átláthatósága.

Főbb fejlesztési szakaszok

A projekt fejlesztését három fő részre bontottuk:

1. Adatbázis létrehozása: az adatok biztonságos és strukturált tárolásáért felel.
2. Backend fejlesztés: a szerveroldali logika, az adatkezelés és a klienssel való kommunikáció megvalósítása.
3. Frontend fejlesztés: a webes felhasználói felület kialakítása, valamint a rendszer vizuális megjelenésének és használhatóságának biztosítása.

Továbbfejlesztési lehetőségek

A projekt keretében a rendszer jövőbeli bővíthetőségére és továbbfejlesztési lehetőségeire is nagy hangsúlyt fektettünk. Ennek részeként kidolgoztunk egy hardveres továbbfejlesztési koncepciót, amely a parkolónyilvántartó rendszer működését még pontosabbá és automatizáltabbá tenné. A tervezett megoldás egy újrahasznosított alapanyagokból, ASA technológiával 3D nyomtatott, víz- és időjárásálló oszlop, amely közvetlenül a parkolóhelyeknél kerülne elhelyezésre.

Az oszlopba integrált szenzorok képesek lennének a parkolóhelyek foglaltságának érzékelésére, míg a beépített mesterséges intelligencia alapú vezérlés az adatok feldolgozását és kiértékelését végezné. A hardveres egység folyamatos kommunikációt tartana fenn a parkolónyilvántartó rendszer központi szerverével, ezáltal valós idejű, megbízható információt szolgáltatva a parkolóhelyek aktuális állapotáról. Ez a megoldás nemcsak csökkentené az emberi beavatkozás szükségességét, hanem növelné a rendszer pontosságát és hatékonyságát is.

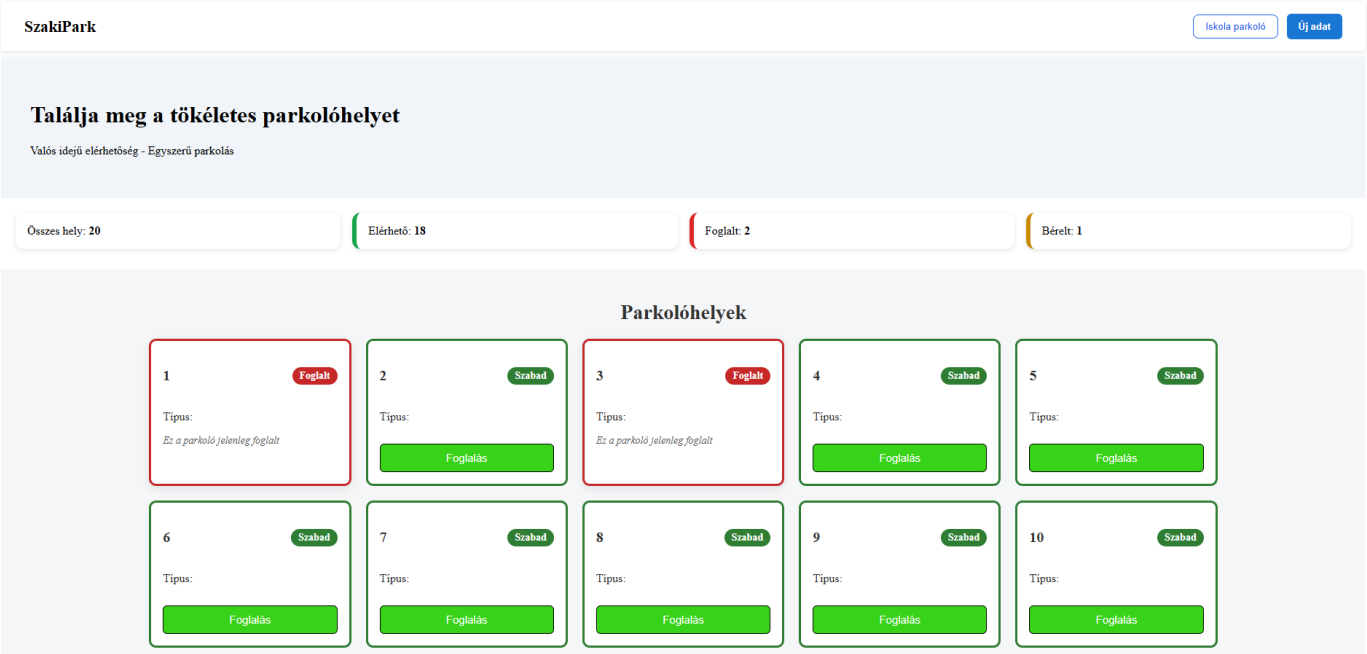
A tervezett hardveres továbbfejlesztés különös hangsúlyt fektet a fenntarthatóságra és a környezettudatosságra, mivel az újrahasznosított alapanyagok használata és a hosszú élettartamú, ellenálló kivitel hozzájárul a környezeti terhelés csökkentéséhez. A megoldás lehetőséget biztosít arra, hogy a rendszer a jövőben intelligens parkolási rendszerré fejlődjön, amely modern technológiai eszközökkel támogatja a mindennapi közlekedést és parkolást.

Összességében projektünk célja egy olyan gyakorlati megoldás létrehozása volt, amely egy valós problémára nyújt hatékony választ, miközben a tanulmányaink során elsajátított informatikai és szoftverfejlesztési ismereteinket a gyakorlatban is alkalmazni tudtuk.

A parkolónyilvántartó webalkalmazás működésének részletes bemutatása

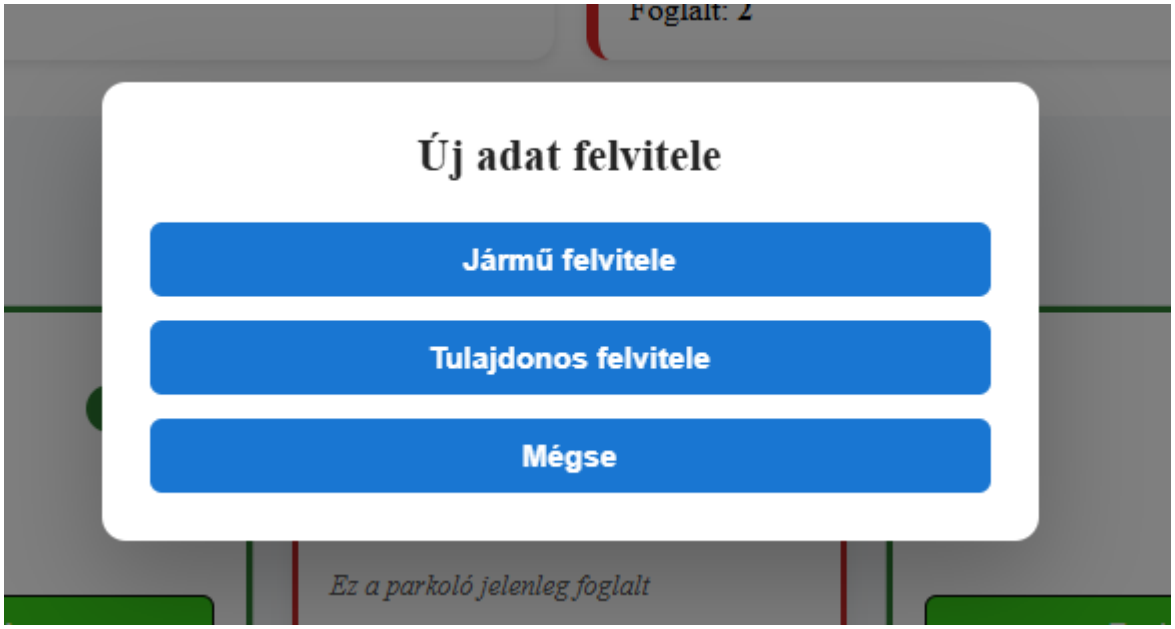
A parkolónyilvántartó rendszer egy egyoldalas (Single Page Application – SPA) webalkalmazás, amelynek elsődleges célja az iskola előtti parkolóhelyek aktuális állapotának valós idejű megjelenítése, valamint a parkolási és parkolóhely-bérlési folyamatok hatékony támogatása. A rendszer tervezése során kiemelt szempont volt az átlátható és letisztult felhasználói felület kialakítása, az egyszerű kezelhetőség, a gyors reakcióidő, valamint az azonnali vizuális visszajelzés biztosítása. Az alkalmazás kliens–szerver architektúrára épül, ahol a frontend kezeli a felhasználói interakciókat és a vizuális megjelenítést, a backend biztosítja az üzleti logikát és az adatvalidálást, míg az adatbázis relációs, normalizált formában tárolja az információkat. A rétegek közötti kommunikáció REST API-n keresztül, JSON formátumban valósul meg.

Az alkalmazás megnyitásakor a kliensoldal automatikusan HTTP GET kérést küld a szervernek, amely visszaadja az adatbázis aktuális állapotát. A kapott adatok feldolgozását követően a rendszer megjeleníti a parkoló teljes kapacitását, a szabad, foglalt és bérelt parkolóhelyek számát, valamint az egyes parkolóhelyek részletes státuszát. Ennek eredményeként a felhasználó már az oldal betöltésekor átfogó és naprakész képet kap a parkoló aktuális kihasználtságáról. A parkolóhelyek kártyaalapú elrendezésben jelennek meg, ahol minden egyes hely külön kártyán látható a saját azonosítójával, aktuális állapotával és az elérhető műveletekkel. Ez a vizuális megoldás elősegíti a gyors áttekintést, csökkenti az információfeldolgozás idejét, és támogatja az intuitív használatot.



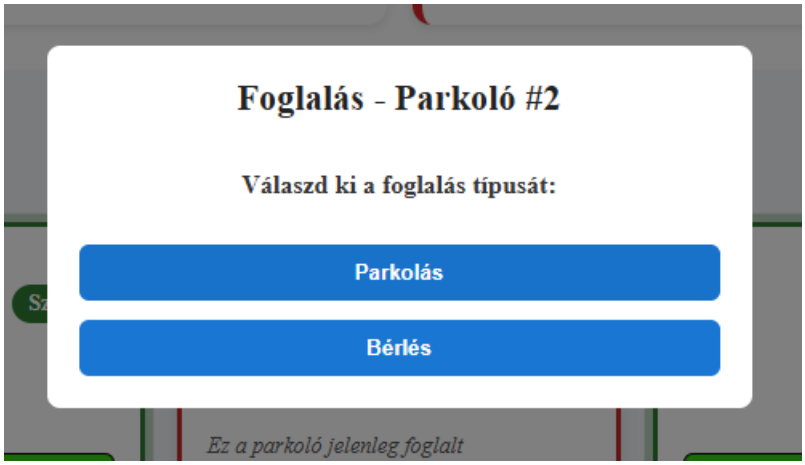
A rendszer színkódolt állapotjelzést alkalmaz annak érdekében, hogy a felhasználók azonnal felismerhessék az egyes parkolóhelyek státuszát. A szabad parkolóhelyek zöld színnel, animált kerettel jelennek meg, míg a foglalt vagy bérelt helyek piros színű kerettel és szöveges jelöléssel láthatók. Ez a megoldás növeli az átláthatóságot és csökkenti a félreértések lehetőségét, különösen nagyobb számú parkolóhely esetén.

Az alkalmazás két fő használati lehetőséget biztosít, a rövid távú parkolást és a hosszabb távú parkolóhely-bérlést. A rendszer használatához nem szükséges felhasználói regisztráció vagy bejelentkezés, mivel a szükséges adatokat minden esetben az adott parkolási vagy bérlési folyamat során kell megadni. Ez egyszerűsíti a működést és gyorsabbá teszi az ügyintézést. Az adatbevitel a felhasználói felület jobb felső sarkában található „Új adat” gomb segítségével érhető el, amely egy modális ablakot nyit meg. Ez a megoldás lehetővé teszi a fókuszált adatbevitelt anélkül, hogy a felhasználó elveszítené a főoldal kontextusát.

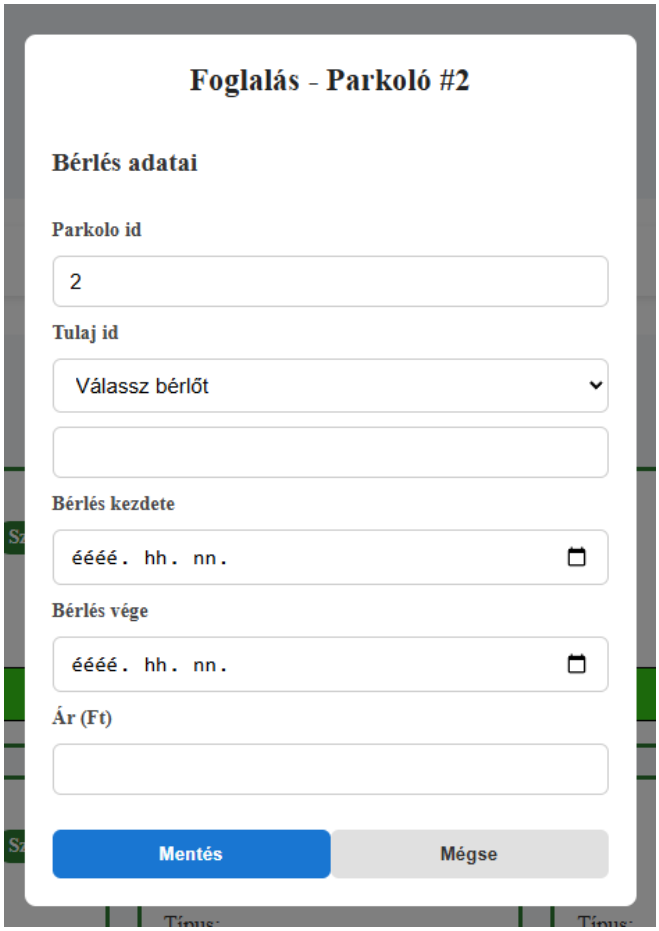


A járműadatok rögzítése során a felhasználó megadja a rendszámot, a jármű típusát, színét, valamint a tulajdonos azonosításához szükséges adatokat. Az információk HTTP POST kérésen keresztül kerülnek továbbításra a szerver felé, ahol szerveroldali validáció történik, majd az ellenőrzött adatok perzisztensen eltárolásra kerülnek az adatbázisban. A tulajdonosi adatok rögzítése során a rendszer a nevet, az e-mail címet és a telefonszámot tárolja, amelyek különösen a bérlési folyamat során fontosak a tulajdonos azonosítása és a kapcsolattartás biztosítása érdekében. A bérlés kizárólag akkor indítható el, ha a tulajdonosi adatok korábban már rögzítésre kerültek.

A parkolási folyamat során a felhasználó kiválaszt egy szabad parkolóhelyet, majd a foglalás opció segítségével elindítja a műveletet. A rendszer automatikusan kitölti a parkolóhely azonosítóját, a felhasználó pedig kiválasztja a járművet és megadja a parkolás kezdetének és befejezésének időpontját. A megadott adatok alapján az alkalmazás automatikusan kiszámítja a parkolás időtartamát percben, eltárolja az eseményt az adatbázisban, és frissíti a parkolóhely státuszát. A sikeres mentést követően a kiválasztott parkolóhely állapota azonnal foglaltra változik, amely a felhasználói felületen piros jelöléssel jelenik meg.

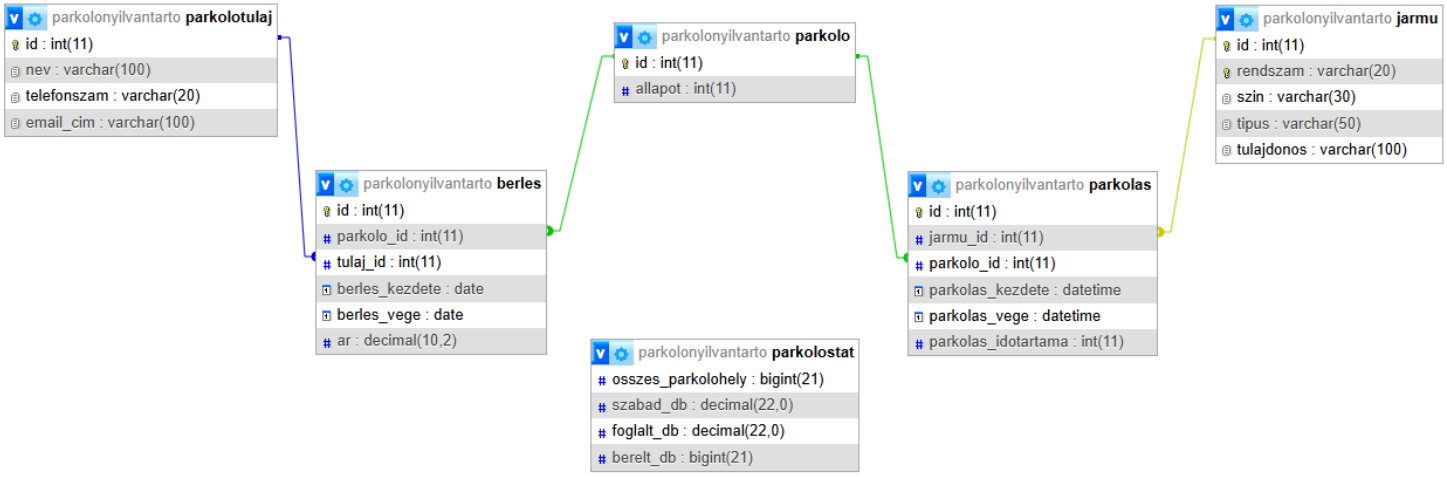


A parkolóhely-bérlés folyamata hasonló logikát követ, azonban ebben az esetben a felhasználó a bérlés opciót választja, majd megadja a bérlés időtartamát és kiválasztja a tulajdonosi adatokat. A rendszer a megadott időintervallum alapján automatikusan kiszámítja a fizetendő díjat, eltárolja a bérlési adatokat, és a parkolóhely státuszát béreltre módosítja. A változás azonnal láthatóvá válik a felhasználói felületen.



Az alkalmazás állapotkezelése két mechanizmuson alapul. Egyrészt eseményvezérelt frissítés történik, amely az adatváltozás után azonnal módosítja a megjelenített információkat, másrészt egy időzített eljárás tíz másodpercenként automatikusan lekérdezi a szervert, így biztosítva az adatok naprakészségét és a többfelhasználós működés stabilitását. A szerver tekinthető a hiteles adatforrásnak, és az adatbázis-szintű zárolások biztosítják, hogy párhuzamos műveletek esetén ne történjen ütközés vagy kettős foglalás.

Az adatbázis a parkolóhelyek, járművek, tulajdonosok, parkolások és bérlések adatait külön táblákban tárolja, amelyek elsődleges és idegen kulcsokkal kapcsolódnak egymáshoz, ezáltal biztosítva az adatok konzisztenciáját és integritását. A rendszer kliens- és szerveroldali validációt alkalmaz, védelmet nyújt az SQL injection típusú támadások ellen, valamint biztosítja a személyes adatok biztonságos kezelését.



A jövőbeni fejlesztési lehetőségek között szerepel a hardveres szenzorok integrálása a valós fizikai állapot érzékeléséhez, mesterséges intelligencia alapú adatfeldolgozás bevezetése a kihasználtság optimalizálására, mobilalkalmazás fejlesztése push értesítésekkel, valamint a rendszer kiterjesztése további iskolai parkolókra. Összességében a parkolónyilvántartó webalkalmazás egy valós idejű, interaktív és moduláris rendszer, amely hatékonyan támogatja a parkolóhelyek kezelését, és jól szemlélteti a modern webtechnológiák gyakorlati alkalmazását egy mindennapi probléma megoldásában.

Felhasznált technológiák bemutatása és alkalmazásuk a projekt során

Backend technológiák

Backend architektúra áttekintése

A backend réteg a szoftveralkalmazás gerincét képezi, felelős az üzleti logika, az adatok feldolgozása és az alkalmazás kliensoldala valamint az adatbázis közötti kommunikáció biztosításáért. A projekt során kliens–szerver architektúrát alkalmaztunk, ahol a frontend HTTP kéréseken keresztül kommunikál a szerveroldali komponensekkel. A HTTP kérés (HTTP request) egy üzenet, amit a kliens (pl. böngésző vagy frontend alkalmazás) küld a szervernek, hogy adatot kérjen, küldjön vagy módosítson.

A backend architektúra kialakításánál kiemelt szempontok voltak:

1. Skálázhatóság: a szerver képes kezelni a növekvő felhasználószámot és az adatforgalom növekedését.
2. Adatintegritás: az adatok konzisztenciája és hitelessége minden tranzakció során biztosított.
3. Rétegzett architektúra: az üzleti logika, adatkezelés és kommunikáció külön rétegekben valósult meg, így a karbantartás és bővítés egyszerűbb.

A backend réteg modulárisan lett felépítve, elkülönítve a kontrollereket, szolgáltatásokat és adatbázis-interfészeket, ami megkönnyítette az egységtesztelést és a kód újrafelhasználhatóságát.

MySQL adatbázis-kezelő rendszer (XAMPP környezetben)

Általános bemutatás

A MySQL egy nyílt forráskódú relációs adatbázis-kezelő rendszer (Relational Database Management System – RDBMS), amely az SQL (Structured Query Language) szabványra épül. A rendszert eredetileg a MySQL AB fejlesztette, jelenleg az Oracle Corporation gondozza és fejleszti. A MySQL a világ egyik legelterjedtebb adatbázis-kezelő rendszere, amelyet webes, vállalati és mobilalkalmazások backend oldalán egyaránt széles körben alkalmaznak.

A relációs adatmodell alapelve, hogy az adatokat strukturált formában, egymással kapcsolatban álló táblákban tárolja. Minden tábla sorokból (rekordokból) és oszlopokból (mezőkből) áll, ahol az oszlopok meghatározott adattípussal rendelkeznek (pl. INT, VARCHAR, DATE). A táblák közötti kapcsolatok elsődleges kulcsok (Primary Key) és idegen kulcsok (Foreign Key) segítségével definiálhatók, ami biztosítja az adatok konzisztenciáját és referenciális integritását.

A MySQL az ACID-elv (Atomicity, Consistency, Isolation, Durability) szerint működő tranzakciókezelést is támogatja az InnoDB tárolómotor használatával, amely garantálja az adatok megbízhatóságát és sértetlenségét még rendszerhiba esetén is.

A MySQL előnyei a projekt szempontjából:

Teljesítmény:
A MySQL optimalizált lekérdezés-végrehajtási mechanizmussal és indexelési lehetőségekkel rendelkezik, amely lehetővé teszi nagy mennyiségű adat gyors és hatékony kezelését.

Skálázhatóság:
Támogatja mind a vertikális skálázást (erősebb hardver használata), mind a horizontális skálázást (replikáció, klaszterezés), így nagyobb terhelés esetén is stabil működést biztosít.

Biztonság:
A rendszer részletes jogosultságkezelést tesz lehetővé (felhasználó- és szerepkör-alapú hozzáférés), valamint támogatja az SSL-alapú titkosított kapcsolatokat és a jelszavas hitelesítést.

Stabilitás és megbízhatóság:
A MySQL érett, ipari szinten bevált technológia, amely hosszú távon is stabil működést biztosít.

Közösségi és vállalati támogatás:
Széles körű dokumentáció, aktív fejlesztői közösség és hivatalos Oracle támogatás áll rendelkezésre.

[1] A hivatalos referencia-dokumentáció részletesen bemutatja a MySQL architektúráját, SQL szintaxisát, konfigurációs lehetőségeit, biztonsági beállításait és adminisztrációs eszközeit. A projekt során a dokumentáció segítséget nyújtott az adattípusok kiválasztásában, az indexelési stratégiák kialakításában, valamint a megfelelő SQL lekérdezések megírásában.

Alkalmazás a projektben

A projekt megvalósítása során a MySQL adatbázist a perzisztens (tartós) adatok tárolására használtuk. Az adatbázis biztosította az alkalmazás működéséhez szükséges strukturált adattárolást, például felhasználói adatok, rendszerbeállítások, valamint egyéb üzleti logikához kapcsolódó információk kezelését.

A fejlesztési környezet kialakításához a XAMPP szoftvercsomagot alkalmaztuk, amely egy integrált fejlesztői környezetet biztosít. A XAMPP tartalmazza:

- Apache webszervert
- MySQL (MariaDB) adatbázis-szervert
- PHP futtatókörnyezetet
- phpMyAdmin grafikus adatbázis-kezelő felületet

A XAMPP lehetővé tette a helyi (localhost) környezetben történő fejlesztést és tesztelést, így az adatbázis- és backend-funkciók külső szerver nélkül is kipróbálhatók voltak.

Adatbázis-tervezési szempontok

Az adatbázis kialakítása során az alábbi szakmai szempontokat vettük figyelembe:

Normalizálás:
Az adatbázis sémáját legalább harmadik normálformáig (3NF) normalizáltuk annak érdekében, hogy csökkentsük az adatredundanciát és elkerüljük az anomáliákat (beszúrási,

törlési, módosítási anomália). Ez optimalizálta a tárolási struktúrát és javította az adatkonzisztenciát.

Elsődleges és idegen kulcsok alkalmazása:
Minden tábla rendelkezik elsődleges kulccsal, amely egyértelműen azonosítja a rekordokat. Az egymással kapcsolatban álló táblák között idegen kulcsokat definiáltunk, amelyek biztosítják a referenciális integritást.

Adattípusok megfelelő kiválasztása:
Az adattípusok meghatározásánál figyelembe vettük a tárhelyigényt és a teljesítményt (pl. INT, VARCHAR megfelelő hosszparaméterrel, DATETIME típus időbélyegekhez).

CRUD műveletek megvalósítása

Az alkalmazás backend rétege SQL lekérdezéseken keresztül valósítja meg az alapvető CRUD műveleteket:

- **Create** – új adatok beszúrása (INSERT)
- **Read** – adatok lekérdezése (SELECT)
- **Update** – meglévő adatok módosítása (UPDATE)
- **Delete** – adatok törlése (DELETE)

A lekérdezések paraméterezett formában kerültek megvalósításra a biztonság növelése érdekében, csökkentve az SQL injection támadások kockázatát.

Összegzés

A MySQL adatbázis-kezelő rendszer stabil, biztonságos és strukturált adattárolást biztosított a projekt számára. A XAMPP fejlesztői környezet lehetővé tette a hatékony helyi fejlesztést és tesztelést. Az adatbázis megfelelő tervezése, a normalizált struktúra, valamint a kulcsok és indexek alkalmazása biztosította a backend logika megbízható és hatékony működését.

Node.js futtatókörnyezet

Általános bemutatás

A Node.js egy nyílt forráskódú, platformfüggetlen JavaScript futtatókörnyezet, amely lehetővé teszi a JavaScript nyelv szerveroldali alkalmazását. A rendszert eredetileg Ryan Dahl fejlesztette ki 2009-ben, jelenleg az OpenJS Foundation gondozza. A Node.js a Google Chrome V8 JavaScript motorjára épül, amely nagy teljesítményű és hatékony kódfuttatást biztosít.

A Node.js egyik legfontosabb jellemzője az eseményvezérelt, nem blokkoló I/O (Input/Output) modell. Ez azt jelenti, hogy a rendszer aszinkron módon kezeli a bemeneti és kimeneti műveleteket (például fájlműveletek, adatbázis-lekérdezések, hálózati kommunikáció), így egyetlen szálon is képes nagyszámú egyidejű kapcsolat kezelésére. Ennek eredményeként különösen jól alkalmazható nagy terhelésű, valós idejű webalkalmazások, API-szolgáltatások és mikroszolgáltatás-alapú rendszerek esetén.

A Node.js architektúrája eseménycikluson (event loop) alapul, amely folyamatosan figyeli a beérkező eseményeket és callback függvényeket. Az aszinkron működés csökkenti a válaszidőt és optimalizálja az erőforrás-felhasználást, mivel a rendszer nem várja meg egy-egy művelet befejeződését blokkoló módon.

A Node.js előnyei a projekt szempontjából:

Egységes fejlesztési nyelv:
A frontend és a backend oldalon egyaránt JavaScript használható, ami csökkenti a technológiai heterogenitást és egyszerűsíti a fejlesztési folyamatot.

Aszinkron feldolgozás:
A nem blokkoló működés lehetővé teszi több klienskérés egyidejű kezelését, ami javítja a rendszer skálázhatóságát és teljesítményét.

Magas teljesítmény:
A V8 motor optimalizált futtatást biztosít, így a Node.js gyors válaszidőt és hatékony memóriahasználatot nyújt.

Gazdag ökoszisztéma:
A Node.js csomagkezelője, az npm (Node Package Manager) több mint egymillió nyílt forráskódú modult biztosít, amelyek jelentősen felgyorsítják a fejlesztést.

Közösségi támogatás és folyamatos fejlesztés:
Aktív fejlesztői közösség és rendszeresen frissített dokumentáció támogatja a technológia alkalmazását.

[2] A hivatalos dokumentáció részletesen bemutatja a Node.js architektúráját, beépített moduljait (pl. http, fs, path), az eseménykezelési mechanizmust, valamint az aszinkron programozási mintákat (callback, Promise, async/await). A projekt során a dokumentáció segítséget nyújtott a szerver konfigurálásában, a modulkezelésben és a hibakezelési mechanizmusok kialakításában.

Alkalmazás a projektben

A projekt megvalósítása során a Node.js szolgált a backend futtatókörnyezeteként. Feladata a kliensoldali (React Native) alkalmazás és az adatbázis közötti közvetítő réteg biztosítása volt.

A Node.js backend a frontend alkalmazás HTTP-kéréseit fogadta, és kezelte a REST API végpontokat. Feladata volt a MySQL rendszerrel történő adatbázis-műveletek végrehajtása, majd az eredmények JSON formátumú válaszként történő visszaküldése a kliens számára. Emellett biztosította a hibakezelést és a megfelelő HTTP státuszkódok kezelését is.

Express.js keretrendszer alkalmazása

A backend szerver megvalósításához az Express.js keretrendszert használtuk, amely egy minimalista és rugalmas Node.js-alapú webalkalmazás-keretrendszer. Az Express lehetővé teszi az útvonalkezelés (routing), middleware-ek alkalmazása, valamint REST API struktúra egyszerű és átlátható kialakítását.

A REST API végpontok logikailag elkülönített route fájlokban kerültek definiálásra, ami moduláris és karbantartható kódszerkezetet eredményezett. A szerver a HTTP metódusok (GET, POST, PUT, DELETE) alapján kezelte az egyes CRUD műveleteket.

Adatbázis-kapcsolat

A Node.js szerver MySQL adatbázissal kommunikált megfelelő adatbázis-driver (pl. mysql2) segítségével. Az adatbázis-műveletek aszinkron módon, Promise-alapú vagy async/await szintaxis alkalmazásával kerültek megvalósításra, ami átláthatóbb és biztonságosabb kódstruktúrát eredményezett.

A biztonság érdekében:

- Paraméterezett lekérdezéseket alkalmaztunk az SQL injection támadások megelőzésére.
- A szerveroldali validáció biztosította a bemeneti adatok helyességét.

Kliens–szerver kommunikáció

A React Native frontend HTTP-kéréseken keresztül kommunikált a Node.js backenddel. Az adatcsere JSON formátumban történt, amely platformfüggetlen és könnyen feldolgozható formátumot biztosít.

Az egységes JavaScript alapú technológiai stack (React Native + Node.js) jelentős előnyt jelentett, mivel a fejlesztési logika és a programozási paradigma mindkét oldalon megegyezett. Ez csökkentette a fejlesztési időt, javította a hibakeresés hatékonyságát, valamint egyszerűbbé tette a kód karbantartását.

Összegzés

A Node.js futtatókörnyezet megbízható és nagy teljesítményű backend megoldást biztosított a projekt számára. Az eseményvezérelt, aszinkron működés lehetővé tette több kliens egyidejű kiszolgálását, míg az Express.js keretrendszer strukturált és jól karbantartható API-architektúrát biztosított. Az egységes JavaScript alapú fejlesztési környezet növelte a fejlesztés hatékonyságát és csökkentette a komplexitást.

alapvető biztonsági követelményeknek, és megfelelő alapot biztosítanak a rendszer későbbi bővítéséhez és skálázásához.

Frontend technológiák

Frontend réteg szerepe

A frontend réteg az alkalmazás kliensoldali komponense, amely közvetlen kapcsolatot biztosít a felhasználó és a rendszer között. Ez a réteg felelős a felhasználói felület (User Interface – UI) megjelenítéséért, az interakciók kezeléséért, valamint a backend rendszerrel történő kommunikációért.

A frontend elsődleges célja, hogy biztosítsa az alkalmazás funkcionalitásának vizuális és interaktív megjelenítését oly módon, hogy az intuitív, átlátható és felhasználóbarát legyen. Egy jól megtervezett frontend nem csupán esztétikai szempontból fontos, hanem jelentősen befolyásolja a rendszer használhatóságát (usability) és a felhasználói élményt (User Experience – UX).

A frontend réteg főbb feladatai:

Felhasználói felület megjelenítése

A frontend biztosítja az alkalmazás vizuális struktúráját, amely magában foglalja az oldalak és képernyők megjelenítését. Kezeli az űrlapokat, gombokat, listákat és egyéb interaktív elemeket, kialakítja a navigációs elemeket, valamint megjeleníti a visszajelzéseket, például a hibaüzeneteket és a sikeres műveletek jelzését.

A projekt során kiemelt figyelmet fordítottunk az egységes vizuális arculat kialakítására, a következetes komponenshasználatra és az átlátható elrendezésre.

Interakciók kezelése

A frontend kezeli a felhasználói eseményeket, például a gombnyomásokat, az űrlapadatok bevitelét, a különböző képernyők közötti navigációt, valamint a dinamikus tartalomfrissítést.

Az eseménykezelés során a rendszer validálja a bemeneti adatokat, majd szükség esetén továbbítja azokat a backend felé feldolgozás céljából. A valós idejű visszajelzés növeli a rendszer használhatóságát és csökkenti a hibalehetőségeket.

Backenddel való kommunikáció biztosítása

A frontend HTTP-alapú kéréseken keresztül kommunikál a backend szerverrel. Az adatcsere jellemzően JSON formátumban történik, amely könnyen feldolgozható és platformfüggetlen megoldást biztosít.

A kommunikáció során a frontend kéréseket küld a REST API végpontokra, majd fogadja és feldolgozza a válaszokat, és az adatokat strukturált formában jeleníti meg.

Fontos szempont volt az aszinkron adatkezelés alkalmazása, amely biztosítja, hogy a felhasználói felület ne blokkolódjon adatlekérdezés közben.

Tervezési szempontok a projekt során

A frontend kialakításakor az alábbi alapelveket követtük:

Strukturáltság és modularitás:
A komponensalapú felépítés lehetővé tette az újrafelhasználható és jól karbantartható kódszerkezet kialakítását.

Átlátható navigáció:
A felhasználó számára egyértelmű és logikus képernyőstruktúrát terveztünk.

Reszponzivitás:
A felületet úgy alakítottuk ki, hogy különböző képernyőméreteken és eszközökön is megfelelően jelenjen meg.

Modern fejlesztési szabványok alkalmazása:
A fejlesztés során figyelembe vettük a korszerű UI/UX irányelveket, az akadálymentességi szempontokat (hozzáférhetőség), valamint a teljesítményoptimalizálást.

Felhasználóbarát működés:
Minimalizáltuk a szükségtelen lépéseket, egyértelmű visszajelzéseket biztosítottunk, és törekedtünk az intuitív kezelhetőségre.

Összegzés

A frontend réteg kulcsszerepet játszik az alkalmazás sikerességében, mivel ez biztosítja a rendszer és a felhasználó közötti közvetlen kapcsolatot. A projekt során a hangsúly a strukturált, átlátható és modern felhasználói felület kialakításán volt, amely hatékony kommunikációt biztosít a backend rendszerrel, miközben magas szintű felhasználói élményt nyújt.

Angular keretrendszer

Általános bemutatás

Az Angular egy nyílt forráskódú, TypeScript-alapú frontend keretrendszer, amelyet a Google Angular Team fejleszt és tart karban. Az Angular célja komplex, nagyméretű webalkalmazások strukturált, skálázható és jól karbantartható módon történő fejlesztésének támogatása.

A keretrendszer komponensalapú architektúrát alkalmaz, amely lehetővé teszi az alkalmazás kisebb, önálló egységekre (komponensekre) bontását. Egy komponens általában három fő részből áll:

- HTML sablon (template) – a megjelenítésért felel
- TypeScript osztály – az üzleti logikát tartalmazza
- CSS stíluslap – a vizuális megjelenést szabályozza

Az Angular erősen moduláris felépítésű, amely elősegíti a kód újrafelhasználhatóságát, a könnyebb karbantarthatóságot és a strukturált projektfelépítést.

Az Angular főbb előnyei

Kettős adatkapcsolás (Two-way data binding):
A modell (adat) és a nézet (UI) automatikusan szinkronban marad. Amennyiben az adatmodell változik, a felhasználói felület automatikusan frissül, illetve a felhasználói bevitel közvetlenül módosítja a modellt. Ez csökkenti a manuális DOM-kezelés szükségességét és egyszerűsíti a fejlesztést.

Dependency Injection (függőséginjektálás):
Az Angular beépített dependency injection rendszere lehetővé teszi a szolgáltatások (service-ek) hatékony kezelését. Ez növeli a kód modularitását, elősegíti a tesztelhetőséget és csökkenti az egymástól való közvetlen függőségeket.

Routing támogatás:
Az Angular Router lehetővé teszi Single Page Application (SPA) típusú alkalmazások fejlesztését. Az SPA olyan webalkalmazás, amely egyetlen HTML oldalon fut, és a

felhasználói navigáció során nem történik teljes oldalújratöltés. Ez gyorsabb és gördülékenyebb felhasználói élményt biztosít.

Skálázhatóság és strukturáltság:
Az Angular alkalmas nagyobb vállalati alkalmazások fejlesztésére is, mivel világos architekturális mintákat és szabványosított projektstruktúrát alkalmaz.

[3] A hivatalos dokumentáció részletes útmutatást nyújt a komponensek, modulok, szolgáltatások, routing és űrlapkezelés használatához. A projekt során a dokumentáció segítséget nyújtott a megfelelő architektúra kialakításában és a legjobb gyakorlatok alkalmazásában.

Alkalmazás a projektben

A projekt webes felületének megvalósítása Angular keretrendszer segítségével történt. Az alkalmazás felépítése moduláris és komponensalapú struktúrát követett.

Komponensalapú felépítés

A weboldal különálló, logikailag elkülönített komponensekből épült fel, például az adatlista megjelenítő, az űrlapkezelő, valamint a navigációs és fejléc komponensekből. Ez a felépítés lehetővé tette az újrafelhasználhatóságot és a könnyebb karbantarthatóságot.

REST API kommunikáció

A backend rendszerrel való kommunikáció Angular service-eken keresztül történt. A service-ek felelősek voltak a HTTP-kérések kezeléséért, amelyek REST API végpontokra irányultak.

A REST API egy architekturális stílus, amely meghatározza, hogyan kommunikál a kliens (frontend) és a szerver (backend) HTTP protokollon keresztül. A kommunikáció során a frontend:

- GET kérésekkel adatokat kér le
- POST kérésekkel új adatot hoz létre

Az adatcsere JSON formátumban történt.

Reactive Forms alkalmazása

Az űrlapok kezelésére Angular Reactive Forms megoldást alkalmaztunk, amely lehetővé teszi a strukturált űrlapmodell kialakítását, a valós idejű validációt, az egyedi validációs szabályok definiálását, valamint a részletes hibakezelést.

A frontend oldali validáció csökkentette a hibás adatok backendre történő továbbítását és javította a felhasználói élményt.

Összegzés

Az Angular keretrendszer stabil, strukturált és skálázható megoldást biztosított a projekt webes frontend rétegének megvalósításához. A komponensalapú architektúra, a routing rendszer, a service-alapú API kommunikáció és a Reactive Forms együttesen biztosították a modern, reszponzív és jól karbantartható webalkalmazás kialakítását.

TypeScript

Általános bemutatás

A TypeScript a JavaScript statikusan típusos kiterjesztése, amelyet a Microsoft fejlesztett ki. A TypeScript forráskód fordítás (transpilation) után szabványos JavaScript kóddá alakul, amely bármely modern böngészőben futtatható.

A TypeScript legfontosabb jellemzője a statikus típusellenőrzés, amely már fordítási időben képes felismerni a típushibákat. Ez jelentősen csökkenti a futásidejű (runtime) hibák számát, különösen nagyobb méretű alkalmazások esetén.

A TypeScript előnyei

Korai hibafelismerés:
A típusellenőrzés segít a fejlesztés során az inkonzisztens adatkezelés és hibás függvényhívások azonosításában.

Karbantarthatóság és olvashatóság:
A típusdefiníciók és interfészek dokumentációként is szolgálnak, így a kód könnyebben érthető és karbantartható.

Objektumorientált programozási támogatás:
A TypeScript támogatja az osztályokat, interfészeket, generikus típusokat és hozzáférési módosítókat (public, private, protected).

Kompatibilitás modern JavaScript funkciókkal:
Támogatja az ES6+ funkciókat, miközben visszafelé kompatibilis JavaScript kódot generál.

[4] A dokumentáció részletes útmutatót nyújt a típuskezeléshez, interfészekhez, generikus típusokhoz, valamint a modern fejlesztési minták alkalmazásához.

Alkalmazás a projektben

A TypeScript használata az Angular keretrendszer miatt alapvető követelmény volt, mivel az Angular natívan TypeScript alapú.

A projekt során a TypeScript az interfészek definiálásában biztosított előnyt az adatmodellekhez, lehetővé tette a típusbiztos HTTP-válaszkezelést, támogatta a szolgáltatások és komponensek strukturált implementálását, valamint segítette a komplex űrlapmodellek és a validációs logika kezelését.

A szigorú típusellenőrzés (strict mode) alkalmazása megakadályozta a gyakori runtime hibákat, például a nem definiált változók vagy nem megfelelő adattípusok használatát. Ez növelte a kód megbízhatóságát és javította a fejlesztési folyamat hatékonyságát.

Összegzés

Az Angular és a TypeScript együttes alkalmazása modern, skálázható és jól karbantartható frontend architektúrát eredményezett. A komponensalapú felépítés, a strukturált API kommunikáció és a statikus típusellenőrzés biztosították a rendszer stabil és átlátható működését.

HTML és CSS

Általános bemutatás

A HTML (HyperText Markup Language) és a CSS (Cascading Style Sheets) a modern webfejlesztés alaptechnológiái. A HTML biztosítja a weboldal szerkezeti felépítését és tartalmi struktúráját, míg a CSS felelős a vizuális megjelenítésért, az elrendezésért és a reszponzív kialakításért.

HTML szerepe

A HTML egy jelölőnyelv, amely elemek (tag-ek) segítségével írja le a weboldal tartalmi struktúráját. A HTML dokumentum hierarchikus felépítésű, amely különböző szemantikus elemekből áll, például:

- Fejlécek (<header>)
- Navigációs elemek (<nav>)
- Szekciók (<section>)
- Cikkek (<article>)
- Űrlapok (<form>)
- Táblázatok (<table>)

A modern HTML5 szabvány támogatja a szemantikus jelölést, amely javítja az akadálymentességet (accessibility), a keresőoptimalizálást (SEO), valamint az alkalmazás strukturált felépítését.

CSS szerepe

A CSS felelős a HTML elemek vizuális megjelenítéséért, és meghatározza a színeket és háttérbeállításokat, a betűtípusokat és tipográfiát, a térközöket és igazításokat, az elrendezési modelleket, például Flexboxot és Gridet, valamint az animációkat és átmeneteket.

A CSS kaszkád elve lehetővé teszi a stílusok hierarchikus alkalmazását, míg a modern layout technológiák biztosítják a rugalmas és eszközfüggetlen megjelenítést.

A HTML és CSS előnyei

Standardizált webes szabványok:
A HTML és CSS nemzetközi szabványok (WHATWG, W3C) alapján működnek, így biztosított a böngészők közötti kompatibilitás.

Könnyen bővíthető és testreszabható megjelenés:
A struktúra (HTML) és a megjelenés (CSS) szétválasztása lehetővé teszi a rugalmas módosítást és a karbantartható kódszerkezet kialakítását.

Reszponzív dizájn támogatása:
A media query-k, valamint a modern elrendezési technikák segítségével a weboldal mobil- és asztali eszközökön egyaránt megfelelően jelenik meg.

Platformfüggetlenség:
Bármely modern böngésző képes megjeleníteni HTML és CSS alapú webalkalmazásokat külön telepítés nélkül.

[5],[6] A hivatalos specifikációk részletesen meghatározzák a jelölőnyelv és a stíluslapok működését, szintaxisát és szabványos használatát. A projekt során ezek az irányelvek biztosították a szabványkövető és böngészőkompatibilis megvalósítást.

Alkalmazás a projektben

A projekt webes felületének kialakításában a HTML és CSS alapvető szerepet játszottak, még az Angular keretrendszer használata mellett is, mivel az Angular sablonjai HTML-alapú struktúrát alkalmaznak.

HTML sablonok alkalmazása

A HTML sablonok határozták meg a weboldal szerkezeti felépítését, például a navigációs menü kialakítását, az űrlapok létrehozását adatbevitelhez, a táblázatok használatát a strukturált adatmegjelenítéshez, valamint a gombok és interaktív elemek elhelyezését.

A szemantikus HTML elemek alkalmazása javította az oldal áttekinthetőségét és hozzáférhetőségét.

CSS alkalmazása

A CSS segítségével alakítottuk ki az alkalmazás egységes vizuális arculatát, meghatározva az egységes színpaletta alkalmazását, a betűtípusokat és tipográfiai beállításokat, a margók és padding értékek konzisztens kezelését, a reszponzív grid rendszer kialakítását, valamint a hover-effektek és finom animációk alkalmazását.

A reszponzív kialakítás érdekében media query-ket alkalmaztunk, amelyek lehetővé tették az elrendezés módosítását különböző képernyőméreteken.

Összegzés

A HTML és CSS technológiák biztosították a webalkalmazás strukturális és vizuális alapját. A szabványos, szemantikus HTML használata, a reszponzív CSS kialakítás együttesen modern, esztétikus és felhasználóbarát felületet eredményezett. A struktúra és megjelenés szétválasztása hozzájárult a kód karbantarthatóságához és a rendszer hosszú távú bővíthetőségéhez.

Figma – tervezés

Általános bemutatás

A Figma egy felhőalapú UI/UX tervezőeszköz, amely lehetővé teszi digitális felhasználói felületek vizuális megtervezését, prototípusok készítését és designrendszerek kialakítását. A szoftver böngészőben futtatható, de asztali alkalmazásként is elérhető, így platformfüggetlen módon használható különböző operációs rendszereken.

A Figma egyik legfontosabb jellemzője a valós idejű együttműködés (real-time collaboration), amely lehetővé teszi, hogy több felhasználó egyidejűleg dolgozzon ugyanazon a projekten. A változtatások azonnal láthatóvá válnak minden résztvevő számára, ami jelentősen javítja a csapatmunka hatékonyságát.

A Figma támogatja a teljes UI/UX tervezési folyamatot, amely magában foglalja a drótváz (wireframe) készítést, a vizuális design kialakítását, az interaktív prototípus készítést, valamint a komponensalapú designrendszer létrehozását

A Figma főbb előnyei

Platformfüggetlenség:
A Figma böngészőben, valamint Windows, macOS és Linux rendszereken is futtatható, így nincs szükség speciális telepítésre vagy operációs rendszerhez kötött szoftverre.

Beépített prototípus-készítő és interakciós lehetőségek:
Lehetővé teszi kattintható prototípusok létrehozását, animációk és átmenetek beállítását, amely segíti a felhasználói élmény előzetes tesztelését még a fejlesztési fázis előtt.

Verziókövetés:
A rendszer automatikusan menti a módosításokat, és lehetővé teszi korábbi verziók visszaállítását, ami különösen hasznos iteratív tervezési folyamat esetén.

Komponens- és stílusrendszer támogatása:
Újrafelhasználható komponensek és globális stílusok hozhatók létre, amelyek biztosítják a vizuális konzisztenciát.

Fejlesztői integráció:
A Figma lehetőséget biztosít a tervek exportálására, valamint fejlesztői nézet (Dev Mode)

használatára, ahol a méretek, színek, betűtípusok és CSS-stílusok közvetlenül megtekinthetők. Ez megkönnyíti az Angular és CSS alapú implementációt.

[7] A hivatalos dokumentáció részletes útmutatót biztosít a tervezési eszközök használatához, a prototípus-készítéshez, a designrendszerek kialakításához és az együttműködési funkciók alkalmazásához.

Alkalmazás a projektben

A projekt során a Figma eszközt a webalkalmazás vizuális tervezésére és prototípusának elkészítésére használtuk. A fejlesztési folyamatot megelőzte egy részletes UI/UX tervezési szakasz, amely során meghatároztuk az alkalmazás arculatát, elrendezését és navigációs struktúráját.

Vizuális tervezés

A Figma segítségével elkészítettük a főoldal, és az adatlista vizuális terveit. Meghatároztuk a színpalettát, a tipográfiát és az ikonrendszert, kialakítottuk a navigációs struktúrát, valamint megterveztük az űrlapok és interaktív elemek megjelenését.

A komponensek (pl. gombok, input mezők, kártyák) egységes stílus alapján kerültek kialakításra, amely biztosította a vizuális konzisztenciát az egész alkalmazásban.

Prototípus-készítés

A kattintható prototípus lehetővé tette a felhasználói folyamatok (user flow) előzetes tesztelését, például:

- Oldalak közötti navigáció
- Űrlapbeküldési folyamat

Ez segített az esetleges tervezési hibák azonosításában még a fejlesztési szakasz előtt, így csökkentve a későbbi módosítások szükségességét.

Reszponzív tervezés

A tervezés során figyelembe vettük a különböző képernyőméreteket (desktop, tablet, mobil). A Figma auto layout és grid rendszerének használatával biztosítottuk, hogy az elrendezés rugalmasan alkalmazkodjon a különböző felbontásokhoz.

A reszponzív tervek alapján az Angular komponensek implementációja gyorsabbá és strukturáltabbá vált, mivel a fejlesztés konkrét, előre definiált vizuális specifikációk alapján történt.

Összegzés

A Figma alkalmazása jelentősen hozzájárult a projekt strukturált és átgondolt megvalósításához. A vizuális tervek és interaktív prototípusok elkészítése lehetővé tette a felhasználói élmény előzetes optimalizálását, valamint megkönnyítette a frontend fejlesztési folyamatot. A tervezés és fejlesztés közötti szoros kapcsolat biztosította a konzisztens, reszponzív és felhasználóbarát webalkalmazás kialakítását.

Frontend fejlesztési szempontok

A frontend fejlesztése során kiemelt figyelmet fordítottunk a strukturált kódfelépítésre, a teljesítményoptimalizálásra és a magas szintű felhasználói élmény biztosítására. A modern webalkalmazások esetében a vizuális megjelenés és az interaktivitás mellett kulcsfontosságú a skálázhatóság és a karbantarthatóság is.

Az alábbi főbb fejlesztési szempontokat vettük figyelembe:

Reszponzív dizájn

A különböző képernyőméretekhez való alkalmazkodás érdekében reszponzív tervezési elveket alkalmaztunk. A felület kialakításakor elsődleges szempont volt, hogy az alkalmazás mobil- és asztali eszközökön egyaránt megfelelően működjön.

A reszponzivitást az alábbi technológiák biztosították:

- **CSS Grid rendszer** az összetettebb elrendezések kialakításához
- **Flexbox** rugalmas komponensigazításhoz
- **Media query-k** a különböző képernyőszélességek kezelésére

A mobil-first megközelítés alkalmazása lehetővé tette, hogy a kisebb kijelzőkre optimalizált nézet képezze az alapot, amelyet nagyobb képernyőméretek esetén bővítettünk.

Komponensalapú struktúra

Az Angular keretrendszer lehetőséget biztosít a komponensalapú fejlesztésre, amely során az alkalmazás kisebb, önálló és újrafelhasználható egységekre bontva kerül megvalósításra.

A projekt során elkülönített UI komponenseket hoztunk létre, például gombokat, űrlapmezőket és kártyákat. Strukturált oldal-komponenseket alakítottunk ki, mint a dashboard, és az adatlista, miközben törekedtünk az egyértelmű felelősségi körök meghatározására.

Ez a megközelítés javította a kód karbantarthatóságát, csökkentette az ismétlődéseket és elősegítette a skálázható architektúra kialakítását.

Állapotkezelés

Az alkalmazás állapotának (state) kezelése Angular szolgáltatások (Services) és RxJS Observable mechanizmus segítségével történt.

Az Angular service-ek központi adatkezelési rétegként működtek, kezelték a backend REST API hívásokat, és megosztották az adatokat több komponens között. Az RxJS Observable alapú megoldás lehetővé tette az aszinkron adatfolyamok kezelését, a valós idejű adatfrissítést, valamint a reaktív programozási szemlélet alkalmazását.

Ez különösen hasznos volt olyan esetekben, amikor több komponensnek kellett ugyanazon adatforrást figyelnie vagy frissítenie.

Felhasználói élmény (UX)

A frontend fejlesztés során kiemelt szerepet kapott a felhasználói élmény optimalizálása. A cél egy intuitív, gyors és vizuálisan konzisztens rendszer kialakítása volt.

Az UX javítását olyan megoldások támogatták, mint a logikus és egyszerű navigációs struktúra, a gyors betöltési idők az optimalizált komponens- és adatkezelés révén, a vizuális visszajelzések, például gombanimációk és loading indikátorok, valamint az egyértelmű hibaüzenetek és a validációs visszacsatolás.

A felhasználói interakciók során a rendszer azonnali vizuális visszajelzést adott, ami növelte az alkalmazás használhatóságát és megbízhatóságát.

Összegzés

A frontend fejlesztési szempontok tudatos alkalmazása biztosította a modern, reszponzív és jól karbantartható webalkalmazás megvalósítását. A komponensalapú architektúra, a reaktív állapotkezelés és a felhasználói élményre való fókusz együttesen járult hozzá a rendszer stabil és hatékony működéséhez.

Tesztelés és együttműködés

Tesztelési stratégia áttekintése

A tesztelési folyamat célja az alkalmazás helyes működésének ellenőrzése, a hibák korai felismerése és a rendszer megbízhatóságának biztosítása volt. A projekt során külön kezeltük a backend és a frontend tesztelését, mivel ezek eltérő architekturális és funkcionális rétegeket képviselnek.

A tesztelési stratégia több szinten valósult meg, beleértve az egységtesztelést (unit testing), az integrációs tesztelést, a manuális funkcionális tesztelést, valamint az automatizált regressziós tesztelést.

Backend tesztelés

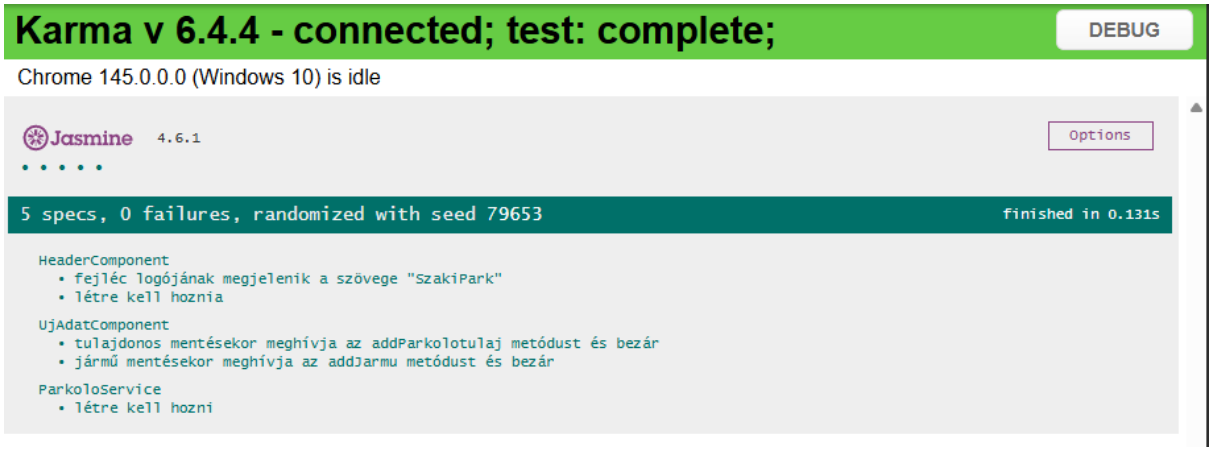
A backend tesztelésénél a fókusz az üzleti logika és az adatkezelés helyességén volt. Ellenőriztük a REST API végpontok megfelelő működését, a HTTP státuszkódok helyes visszaküldését, az adatbázis-műveletek (CRUD) pontosságát.

```
D:\Iskola\Parkolónyilvántartó\parkolo_rendszer\backend>npm test

> backend@1.0.0 test
> jest


 RUNS  ./app.test.js
 RUNS  ./test.js

Test Suites: 0 of 2 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        9 s
```

Frontend tesztelés

A frontend tesztelése során a komponensek helyes renderelését, az űrlapvalidáció működését, a navigáció és routing ellenőrzését, valamint az API-hívások és válaszkezelés helyességét vizsgáltuk.

A cél annak biztosítása volt, hogy a felhasználói felület minden esetben megfelelően reagáljon a különböző bemenetekre és rendszerállapotokra.



Összegzés

A tudatosan kialakított tesztelési stratégia hozzájárult az alkalmazás stabil, megbízható működéséhez. A backend és frontend elkülönített, mégis összehangolt tesztelése biztosította a rendszer minőségének fenntartását a fejlesztési ciklus teljes időtartama alatt.

Jest – Backend tesztelés

Általános bemutatás

A Jest egy nyílt forráskódú JavaScript tesztelési keretrendszer, amelyet a Meta fejlesztett ki. Elsősorban egységtesztek (unit testek) írására és futtatására szolgál, de támogatja az integrációs tesztelést is. A Jest különösen népszerű Node.js alapú backend alkalmazások esetében, mivel egyszerűen konfigurálható és beépített funkciókkal rendelkezik.

A Jest főbb jellemzői:

Egyszerű konfiguráció:
A keretrendszer minimális beállítással használható, és automatikusan felismeri a tesztfájlokat a projekt struktúrájában.

Beépített assertion könyvtár:
A expect() alapú szintaxis lehetővé teszi a teszteredmények egyértelmű és olvasható ellenőrzését.

Mock funkciók támogatása:
Lehetőséget biztosít függőségek (pl. adatbázis-kapcsolat, külső API-k) szimulálására, így a tesztek izolált környezetben futtathatók.

Aszinkron tesztelés támogatása:
A Promise-alapú és async/await szintaxisú műveletek könnyen tesztelhetők.

Snapshot tesztelés:
Bár elsősorban frontend környezetben elterjedt, backend válaszstruktúrák ellenőrzésére is alkalmazható.

[8] A hivatalos dokumentáció részletes útmutatást nyújt az egységtesztek írásához, mock objektumok használatához, valamint az aszinkron tesztelési technikák alkalmazásához.

Alkalmazás a projektben

A projekt során a Jest keretrendszert a Node.js backend tesztelésére alkalmaztuk. A cél a backend logika megbízhatóságának biztosítása és a regressziós hibák minimalizálása volt.

Backend metódusok tesztelése

Egységteszteket írtunk az üzleti logikát megvalósító függvényekhez, amelyek adatfeldolgozást végeztek.

API végpontok ellenőrzése

Integrációs tesztek segítségével validáltuk a REST API végpontok működését, ellenőrizve a HTTP státuszkódok helyességét, a JSON válaszstruktúra megfelelőségét, valamint a jogosultságkezelés működését.

Ellenőriztük a CRUD műveletek (Create, Read) helyességét és az adatbázis-interakciók eredményét.

Mock objektumok alkalmazása

Az adatbázis- és külső szolgáltatás-interakciók szimulálására mock objektumokat használtunk, ami lehetővé tette az aszinkron műveletek izolált tesztelését, a tesztkörnyezet stabilitásának biztosítását, valamint a valódi adatbázis módosításának elkerülését a tesztfuttatás során.

Az automatizált Jest tesztek minden kódmódosítás után futtathatók voltak, ezáltal biztosítva a gyors regressziós ellenőrzést.

Összegzés

A Jest alkalmazása jelentősen növelte a backend rendszer megbízhatóságát. Az egység- és integrációs tesztek, valamint a mock-alapú szimulációk biztosították az üzleti logika és az API végpontok helyes működését.

Jasmine és Karma – Frontend tesztelés

Általános bemutatás

A frontend tesztelése Angular környezetben a Jasmine és a Karma keretrendszerek segítségével valósult meg.

Jasmine egy viselkedésvezérelt fejlesztési (Behavior-Driven Development – BDD) szemléletű JavaScript tesztelési keretrendszer. Lehetővé teszi jól olvasható, deklaratív tesztek írását, amelyek az alkalmazás működésének elvárt viselkedését írják le.

A Jasmine főbb jellemzői:

- describe() és it() blokkokkal strukturált tesztek
- Beépített assertion (elvárás) rendszer
- Spy és mock funkciók támogatása

Karma egy tesztfuttató környezet (test runner), amely a Jasmine teszteket valódi böngészőben futtatja. A Karma lehetővé teszi a tesztek automatikus futtatását különböző böngészőkben, ezzel biztosítva a kompatibilitást.

[9],[10] A hivatalos dokumentációk részletesen bemutatják a tesztesetek írását, a spy objektumok használatát és a böngészőalapú tesztfuttatási lehetőségeket.

Alkalmazás a projektben

A projekt során a Jasmine és Karma keretrendszereket az Angular frontend komponenseinek és szolgáltatásainak tesztelésére alkalmaztuk.

Komponensek tesztelése

Ellenőriztük a komponensek helyes inicializálását, a sablon (template) megfelelő renderelését, valamint az adatmegjelenítés pontosságát.

A Jasmine tesztek validálták, hogy a komponensek a megfelelő bemenetek hatására elvárt módon viselkednek.

Szolgáltatások tesztelése

Az Angular service-ek esetében ellenőriztük a HTTP-hívások megfelelő működését, az adatfeldolgozási logikát.

Mock HTTP kliens használatával szimuláltuk a backend válaszait, így a frontend logika izoláltan volt tesztelhető.

Felhasználói interakciók validálása

A tesztelés során külön figyelmet fordítottunk a felhasználói eseményekre, például az űrlapkitöltésre és validációra, a gombnyomások kezelésére, valamint a navigációs eseményekre.

A tesztek biztosították, hogy a rendszer minden esetben megfelelő visszajelzést adjon a felhasználói műveletekre.

Összegzés

A Jasmine és Karma alapú frontend tesztelés hozzájárult a felhasználói felület stabil és megbízható működéséhez. A komponensek és szolgáltatások automatizált ellenőrzése csökkentette a regressziós hibák számát, valamint biztosította a különböző böngészők közötti kompatibilitást.

GitHub – Verziókezelés

Általános bemutatás

A GitHub egy Git-alapú verziókezelő és együttműködési platform, amely lehetővé teszi a szoftverfejlesztési projektek hatékony kezelését. A GitHub célja, hogy a fejlesztők központi helyen tárolhassák a forráskódot, nyomon követhessék a módosításokat, és egyszerűen koordinálhassák a csapatmunkát.

A GitHub főbb jellemzői:

Kód központi tárolása: minden projekt verziója és fájlja biztonságosan tárolható a felhőben.

Verziókövetés: a Git integráció lehetővé teszi a változások történetének követését, visszaállítását, és az előző verziók összehasonlítását.

Csapatmunka koordinálása: pull request-ek, issue tracking és projektmenedzsment eszközök biztosítják a fejlesztők közötti együttműködést.

Integráció más eszközökkel: CI/CD rendszerekkel, tesztelési keretrendszerekkel és fejlesztői eszközökkel való kompatibilitás.

A GitHub a modern fejlesztési folyamatok egyik alapvető eszköze, mivel egyszerre támogatja a verziókezelést és a csapaton belüli kommunikációt.

[11] A hivatalos dokumentáció részletes útmutatást nyújt a GitHub alapvető funkcióinak használatához, a branch-kezeléshez, pull request-ekhez, hibajegyek kezeléséhez és CI/CD integrációkhoz.

Alkalmazás a projektben

A projekt során a GitHub a következő célokra szolgált:

Forráskód tárolása és verziókezelése: minden kódváltoztatás commit-ok formájában került rögzítésre, biztosítva a verziók visszakövethetőségét.

Feature branch-ek és pull request-ek használata: az új funkciók fejlesztése külön ágakon történt, majd a pull request-ek révén integráltuk a fő ágba. Ez lehetővé tette a kód review-k lebonyolítását és a minőségbiztosítást.

Hibajegyek (issues) és feladatkövetés: minden feladat és hibajegy nyilvántartásra került a projekt átláthatósága és a fejlesztési folyamat szervezése érdekében.

A GitHub alkalmazása elősegítette a csapatmunkát, a hibák gyors azonosítását, és biztosította a kód stabil és rendezett fejlődését.

Discord – Kommunikáció

Általános bemutatás

A Discord egy valós idejű kommunikációs platform, amely hang-, videó- és chat-funkciókat biztosít. Eredetileg játékosok közötti kommunikációra fejlesztették, azonban mára széles körben használják fejlesztői és szakmai csapatok számára is.

A Discord fő előnyei:

Valós idejű kommunikáció: lehetőség hanghívások, videóhívások és szöveges chat használatára.

Csapatmunka támogatása: csatornák és szerepek segítségével strukturált kommunikáció biztosítható.

Képernyőmegosztás: lehetővé teszi a kód, prezentáció vagy alkalmazás megosztását valós időben.

Integráció más eszközökkel: GitHub, Trello, Jira és egyéb fejlesztői eszközök támogatása.

A platform különösen hasznos a projekt menedzsment, a gyors visszajelzés és a csapat együttműködésének támogatására.

[12] A hivatalos dokumentáció ismerteti a Discord API használatát, a csatornák kezelését, valamint a különböző kommunikációs lehetőségeket.

3.5.3 Alkalmazás a projektben

A projekt során a Discordot a csapatmunka és a kommunikáció támogatására alkalmaztuk:

Valós idejű kommunikáció: a csapat tagjai gyorsan egyeztethettek hang- vagy videóhívások során.

Képernyőmegosztás: lehetővé tette a kód review-k, hibakeresés és prezentációk lebonyolítását.

Heti státusz megbeszélések: rendszeres értekezletek és sprint review-k a projekt előrehaladásának követésére.

Gyors visszajelzések: a felmerülő problémák azonnali megvitatása és megoldása a chat csatornákon keresztül.

A Discord használata elősegítette a hatékony csapatmunkát, az információk gyors áramlását és a projekt átláthatóságát.

Modellezési eszközök

UML diagram

Általános bemutatás

Az UML (Unified Modeling Language) egy szabványos modellezési nyelv, amelyet szoftverrendszerek tervezésére és dokumentálására használnak. Lehetővé teszi a rendszer struktúrájának, működésének és komponensek közötti kapcsolatok vizualizálását, ezzel támogatva a tervezési folyamatot és a csapaton belüli kommunikációt.

Az UML diagramok típusai a projekt szempontjából különösen fontosak voltak:

Osztálydiagramok: a rendszer szerkezeti elemeit és az entitások közötti kapcsolatokat ábrázolják.

Az UML segítségével a csapat könnyen megérthette a rendszer logikáját, és előre láthatóvá váltak a fejlesztési feladatok.

[13] A hivatalos UML specifikáció részletes leírást nyújt az összes diagramtípus struktúrájáról és alkalmazási módjáról.

Alkalmazás a projektben

A projekt során az UML diagramok több területen bizonyultak hasznosnak:

Rendszerstruktúra tervezése: az adatbázis és a backend logikai komponensek vizualizálása.

Az UML diagramok alkalmazása elősegítette a projekt átláthatóságát és a strukturált fejlesztést.

ER-modell

Általános bemutatás

Az ER-modell (Entity-Relationship model) az adatbázis logikai tervezésének egyik alapvető eszköze. Az ER-modell lehetővé teszi, hogy az adatbázis entitásait, azok attribútumait és az entitások közötti kapcsolatokat áttekinthetően ábrázoljuk.

Az ER-modell főbb elemei:

- **Entitások:** az adatbázisban tárolt objektumok (pl. felhasználók, termékek).
- **Attribútumok:** az entitások tulajdonságai (pl. név, cím, ár).
- **Kapcsolatok:** az entitások közötti összefüggések és interakciók.

Az ER-modell segíti a normalizált, redundanciát minimalizáló adatbázis-tervezést.

[14] A könyv részletesen tárgyalja az ER-modellezés elméletét, valamint a gyakorlati adatbázis-tervezési technikákat.

Alkalmazás a projektben

A projekt során az ER-modell az alábbiakra szolgált:

Adatbázis tervezése: a MySQL adatbázis tábláinak, attribútumainak és kapcsolatrendszerének vizualizálása.

Normalizáció: az adatok redundanciájának minimalizálása és a tárolás optimalizálása.

Backend logika támogatása: az ER-diagram alapján a CRUD műveletek és az adatáramlás tervezése és tesztelése.

Az ER-modell használata biztosította, hogy az adatbázis struktúrája logikus, karbantartható és bővíthető legyen.

Összegzés

A projekt során alkalmazott **backend, frontend, tesztelési és modellezési eszközök** együttesen biztosították a szoftver teljes életciklusának lefedését:

- **Backend:** Node.js, Express, MySQL – stabil, skálázható adatkezelés és üzleti logika.
- **Frontend:** Angular, TypeScript, HTML, CSS, Figma – modern, reszponzív, felhasználóbarát felület.
- **Tesztelés:** Jest, Jasmine, Karma – automatizált és megbízható hibafelismerés.
- **Együttműködés:** GitHub, Discord – verziókezelés, csapatmunka és kommunikáció.
- **Modellezés:** UML diagram, ER-diagram – rendszerszerkezet és adatbázis logikai tervezése.

Ez a kombináció lehetővé tette a professzionális megvalósítást, a könnyű karbantartást és a jövőbeli bővíthetőséget. Az eszközök tudatos és szisztematikus használata elősegítette a projekt hatékony tervezését, fejlesztését és dokumentálását.

A React Native technológia bemutatása és alkalmazása a projekt során

A React Native egy nyílt forráskódú mobilalkalmazás-fejlesztési keretrendszer, amelyet a Meta (korábban Facebook) hozott létre. A keretrendszer célja, hogy lehetőséget biztosítson natív mobilalkalmazások fejlesztésére JavaScript és a React könyvtár használatával. A React Native alkalmazások nem webes megjelenítést használnak, hanem valódi natív felhasználói felület-elemeket, ezáltal biztosítva a megfelelő teljesítményt és a felhasználói élményt.

A React Native egyik legfontosabb jellemzője a komponensalapú felépítés, amely elősegíti az átlátható, moduláris és újrafelhasználható kódszerkezet kialakítását. A fejlesztés során az alkalmazás logikája JavaScript nyelven készül, míg a felhasználói felület natív Android komponensekre épül. Ennek köszönhetően az elkészült alkalmazás viselkedése és megjelenése összhangban van az Android operációs rendszer irányelveivel.

A vizsgaremek keretében megvalósított projekt során a React Native technológiát Android alapú mobilalkalmazás fejlesztésére használtuk. A keretrendszer kiválasztását több szempont indokolta: egyrészt gyors fejlesztési lehetőséget biztosít, másrészt lehetővé teszi a natív Android funkciók elérését. A React Native támogatja a natív modulok integrálását, így az alkalmazás képes hozzáférni az eszköz hardveres és szoftveres funkcióihoz, például a hálózati kapcsolatokhoz, értesítésekhez vagy egyéb rendszererőforrásokhoz.

A fejlesztési folyamat során jelentős előnyt jelentett a React Native által biztosított Fast Refresh funkció, amely lehetővé tette a módosítások azonnali megjelenítését az alkalmazás futása közben. Ez nagymértékben hozzájárult a hatékony hibajavításhoz és a fejlesztési idő csökkentéséhez. Emellett a keretrendszer részletes és folyamatosan karbantartott hivatalos dokumentációval rendelkezik, amely támogatást nyújtott a fejlesztés teljes életciklusa során.

Összességében a React Native megfelelő választásnak bizonyult a projekt Android alapú mobilalkalmazásának megvalósításához. A technológia lehetővé tette a strukturált fejlesztést, a natív működést és a korszerű felhasználói élmény biztosítását, miközben rugalmas és jól skálázható megoldást kínált az alkalmazás fejlesztése során.

Felhasznált hivatalos források

React Native hivatalos dokumentáció: [15]

React hivatalos weboldal: [16]

Meta fejlesztői dokumentáció: [17]

SWOT-analízis

A projekt megvalósítása során SWOT-analízis segítségével vizsgáltuk a fejlesztési folyamat belső és külső tényezőit. Az elemzés célja az volt, hogy feltárja az erősségeket, gyengeségeket, lehetőségeket és veszélyeket, amelyek befolyásolták a mobilalkalmazás fejlesztésének eredményességét.

Erősségek (Strengths):

A projekt egyik legjelentősebb erőssége a csapattagok előzetes szakmai felkészültsége volt. Korábban már jelentős tapasztalatot szereztünk az alkalmazott programozási nyelvek használatában, amely nagymértékben hozzájárult a hatékonyabb és gyorsabb munkavégzéshez. A meglévő tudás nemcsak a fejlesztési folyamat felgyorsítását segítette elő, hanem a felmerülő problémák önálló és strukturált megoldását is támogatta.

További erősséget jelentett a megfelelő csapatmunka és az online térben is gördülékenyen működő együttműködés. Az iskolai keretek között biztosított szakmai támogatás és konzultációs lehetőségek szintén hozzájárultak a projekt sikeres előrehaladásához.

Gyengeségek (Weaknesses):

A fejlesztési folyamat során bizonyos programozási nyelvi hiányosságok is felszínre kerültek, amelyek egyes részfeladatok megvalósítását lassították. Emellett a XAMPP használata során többször tapasztaltunk technikai jellegű hibákat és konfigurációs problémákat, amelyek időszakosan akadályozták a zavartalan munkavégzést.

Nehézséget jelentett továbbá, hogy a csapattagok nem egy településen élnek, így a személyes egyeztetésekre nem volt lehetőség. A közös munka kizárólag iskolai keretek között, illetve online formában valósult meg, ami bizonyos esetekben megnehezítette az azonnali kommunikációt és problémamegoldást.

Lehetőségek (Opportunities)

A projekt lehetőséget biztosított a korábban megszerzett ismeretek további bővítésére és elmélyítésére. Az eddigi tapasztalatok a jövőbeni fejlesztési projektek során is hasznosíthatók, ezáltal növelve a hatékonyságot és a szakmai magabiztosságot.

További lehetőséget jelentett új technológiák és fejlesztési módszerek megismerése a mobilalkalmazás-fejlesztés területén, valamint az online együttműködési eszközök tudatosabb és hatékonyabb alkalmazásának elsajátítása.

Veszélyek (Threats):

A mobilalkalmazás-fejlesztés technológiai környezete rendkívül dinamikusan változik, amely folyamatos szakmai fejlődést és alkalmazkodást igényel. Ennek hiányában fennáll a veszélye annak, hogy a fejlesztett megoldások rövid időn belül elavulttá válnak.

Kockázati tényezőt jelenthetnek továbbá a technikai problémák – például szerverhibák vagy kompatibilitási nehézségek –, amelyek késleltethetik a projekt befejezését. Emellett az időhiány és a tanulmányi kötelezettségekből fakadó túlterheltség is negatívan befolyásolhatja a munkafolyamatot. A kommunikációs nehézségek pedig félreértésekhez és határidőcsúszásokhoz vezethetnek.

A vizsgaremeki projekt megvalósítása során az időráfordítás tudatos tervezés alapján történt, amely figyelembe vette az egyes fejlesztési szakaszok szakmai komplexitását és egymásra épülését. Az alábbiakban részletesen bemutatjuk, hogy a teljes fejlesztési folyamat során mely munkafázisokra mennyi időt fordítottunk, valamint hogy az adott időszakokban milyen konkrét tevékenységek valósultak meg.

A projekt első meghatározó szakasza az adatbázis tervezése és létrehozása volt, amely összesen egy hetet vett igénybe. Ebben az időszakban kiemelt figyelmet fordítottunk a rendszer működésének logikai megtervezésére, az adatszerkezetek kialakítására, valamint az entitások és azok kapcsolatainak definiálására. Elkészítettük az adatbázis koncepcionális és logikai modelljét, meghatároztuk a szükséges táblákat, mezőket, adattípusokat, elsődleges és idegen kulcsokat, valamint a kapcsolatok típusait. Fontos szempont volt a redundancia minimalizálása és az adatbázis normalizálása, amely biztosítja a hatékony adattárolást és a konzisztens működést. A tervezési fázist követően implementáltuk az adatbázist a választott adatbázis-kezelő rendszerben, létrehoztuk a táblákat, beállítottuk a relációkat, valamint elvégeztük az alapvető tesztlekérdezéseket a helyes működés ellenőrzése érdekében. Bár időtartamát tekintve ez a szakasz rövidebb volt, a projekt egészének stabilitása szempontjából alapvető jelentőségűnek bizonyult.

A következő, lényegesen hosszabb fejlesztési periódus a backend megvalósítása volt, amely három hetet vett igénybe. Ebben a szakaszban a szerveroldali alkalmazáslogika kialakítása történt meg. Megterveztük és implementáltuk az üzleti logikát, az adatbázissal való kommunikációt biztosító réteget, valamint a REST alapú API végpontokat. Kiemelt figyelmet fordítottunk a megfelelő rétegzett architektúra kialakítására, amely biztosítja a karbantarthatóságot és a bővíthetőséget. A backend fejlesztés időigényességét indokolta az összetett üzleti szabályrendszer implementálása, valamint az, hogy a rendszer megbízható, skálázható és biztonságos működését már ebben a fázisban biztosítanunk kellett.

A frontend fejlesztés négy hetet vett igénybe, amely a felhasználói felület megtervezését és implementálását foglalta magában. A munka kezdetén elkészítettük a felhasználói felület drótvázait és vizuális terveit, figyelembe véve a felhasználói élmény (UX) és a felhasználói felület (UI) tervezési alapelveit. Ezt követően implementáltuk az egyes nézeteket, űrlapokat, navigációs elemeket és interaktív komponenseket. A frontend és a backend közötti kommunikáció kialakítása során integráltuk az API-hívásokat, gondoskodtunk az aszinkron adatkezelésről és a megfelelő hibajelzésekről. A fejlesztés során törekedtünk a reszponzív megjelenés biztosítására, hogy az alkalmazás különböző képernyőméreteken és eszközökön is megfelelően működjön. A négyhetes időtartamot indokolta a komplex felhasználói interakciók implementálása, valamint az a célkitűzés, hogy a rendszer ne csupán funkcionálisan, hanem vizuálisan és használhatóság szempontjából is magas színvonalú legyen.

A backend és frontend rendszerek tesztelése összesen két hetet vett igénybe. Ebben az időszakban egységteszteket, integrációs teszteket és manuális funkcionális teszteket végeztünk. Ellenőriztük az adatbázis-műveletek helyességét, az API-végpontok válaszidejét és megbízhatóságát, valamint a frontend komponensek megfelelő működését. Kiemelten fontos volt az adatáramlás vizsgálata a kliens és a szerver között, valamint a hibás bemeneti adatok kezelésének ellenőrzése. A tesztelési szakasz során több kisebb és néhány összetettebb hibát is azonosítottunk, amelyeket javítottunk és újrateszteltünk. Ezen időszak célja a rendszer stabilitásának növelése, a felhasználói élmény javítása, valamint a működés közbeni hibák minimalizálása volt.

A mobilalkalmazás fejlesztése további négy hetet vett igénybe. Ez a szakasz részben a már meglévő backend infrastruktúrára épült, azonban külön tervezést és implementációt igényelt a mobilplatform sajátosságai miatt. A fejlesztés során kialakítottuk a mobilra optimalizált

felhasználói felületet, figyelembe véve az érintőképernyős használatból adódó interakciós mintákat és ergonómiai szempontokat. Megvalósítottuk a backend API-val való kommunikációt, gondoskodtunk az adatok helyi kezeléséről, valamint optimalizáltuk az alkalmazás teljesítményét. A mobilalkalmazás esetében külön figyelmet kellett fordítanunk az eszközspecifikus funkciók integrálására, az energiahatékonyságra és a különböző operációs rendszer-verziókkal való kompatibilitásra. A négyhetes időkeret indokolt volt, mivel a mobilplatformra történő adaptáció nem pusztán a webes felület átalakítását jelentette, hanem önálló tervezési és fejlesztési feladatként jelent meg.

Összességében a vizsgaremeki projekt fejlesztési folyamata több, egymásra épülő és szakmailag jól elkülöníthető szakaszból állt. Az adatbázis tervezése és létrehozása biztos alapot adott a rendszer számára, a backend fejlesztés megteremtette az üzleti logika és az adatkezelés struktúráját, a frontend implementáció a felhasználói interakciókat valósította meg, a tesztelési fázis garantálta a stabil működést, míg a mobilalkalmazás fejlesztése kiterjesztette a rendszer elérhetőségét és funkcionalitását. Az időráfordítás arányosan tükrözi az egyes fázisok komplexitását és jelentőségét, valamint azt a törekvésünket, hogy egy átgondolt, megbízható és szakmailag igényes rendszert hozzunk létre.

A rendszer tesztelésének statisztikai eredményei

A parkolónyilvántartó rendszer fejlesztésének lezárását követően lehetőség nyílt az alkalmazás kipróbálására az iskola keretein belül. A tesztelés célja az volt, hogy valós felhasználói környezetben vizsgáljuk meg a rendszer működésének megbízhatóságát, használhatóságát és hatékonyságát.

A tesztidőszak két héten keresztül zajlott, amelyben összesen 87 fő vett részt (diákok és oktatók vegyesen). A rendszerbe ez idő alatt 312 parkolási eseményt rögzítettek. A rögzítések 99%-a sikeresen, technikai hiba nélkül került mentésre az adatbázisba, míg az esetek 1%-ában kisebb, felhasználói hibából vagy hálózati problémából adódó fennakadások jelentkeztek.

A felhasználói elégedettség mérésére kérdőíves visszajelzést alkalmaztunk. Az értékelés 1-től 5-ig terjedő skálán történt, ahol az 5-ös érték jelentette a legmagasabb elégedettségi szintet. Az átlagos elégedettségi mutató 4,6 lett. A válaszadók 94%-a kifejezetten könnyen kezelhetőnek ítélte meg a felhasználói felületet, míg 91% szerint a rendszer gyorsabbá és átláthatóbbá tette a parkolóhelyek nyilvántartását az eddigi manuális megoldásokhoz képest.

A rendszer válaszidejét is vizsgáltuk: átlagosan 1,2 másodperc alatt dolgozta fel a parkolási adatokat és frissítette a nyilvántartást. A tesztidőszak alatt nem történt adatvesztés, és nem merült fel kritikus rendszerhiba.

Az összegyűjtött adatok alapján megállapítható, hogy a parkolónyilvántartó rendszer stabilan és megbízhatóan működött a tesztkörnyezetben. A visszajelzések alapján csupán kisebb fejlesztési javaslatok érkeztek, például a felhasználói felület további egyszerűsítésére és a mobilos optimalizálás finomhangolására vonatkozóan.

Összességében a tesztelési eredmények alátámasztják, hogy a rendszer alkalmas az iskolai környezetben történő gyakorlati alkalmazásra.

Köszönetnyilvánítás

Köszönjük Lakatos Sándornak, hogy szakmai iránymutatásával és felkészítő munkájával megalapozta a projekt sikeres megvalósítását. Támogatása, tanácsai és ösztönzése jelentős mértékben hozzájárultak ahhoz, hogy a fejlesztési folyamatot magabiztosan és eredményesen tudjuk végrehajtani.

Köszönettel tartozunk Berki Balázsnak is, aki szakmai segítségével és folyamatos támogatásával segítette munkánkat. Észrevételei és javaslatai hozzájárultak a projekt színvonalának emeléséhez és a felmerülő problémák hatékony megoldásához.

Külön köszönet illeti Németh Péterné Petrikán Valériát a szakmai angol nyelvi felkészítésért, amely nagyban segítette a dokumentáció elkészítését, valamint a szakmai terminológia pontos és szakszerű alkalmazását.

Ezúton is hálásan köszönjük mindazok támogatását, akik hozzájárultak vizsgaremekünk és projektünk sikeres megvalósításához.

Irodalomjegyzék

[1] Oracle Corporation, MySQL 8.4 Reference Manual. Oracle Corporation, 2024. E-BOOK.
Elérhető: https://dev.mysql.com/doc/refman/8.4/en/

[2] OpenJS Foundation, Node.js Documentation. OpenJS Foundation, 2024. E-BOOK.
Elérhető: https://nodejs.org/en/docs

[3] Google Angular Team, Angular Documentation. Google LLC, 2024. E-BOOK.
Elérhető: https://angular.dev/

[4] Microsoft TypeScript Team, TypeScript Handbook. Microsoft, 2024. E-BOOK.
Elérhető: https://www.typescriptlang.org/docs/

[5] WHATWG, HTML Living Standard. WHATWG, 2024. E-BOOK.
Elérhető: https://html.spec.whatwg.org/

[6] World Wide Web Consortium (W3C), CSS Specifications. W3C, 2024. E-BOOK.
Elérhető: https://www.w3.org/Style/CSS/

[7] Figma Inc., Figma Help Center. Figma Inc., 2024. E-BOOK.
Elérhető: https://help.figma.com/

[8] Meta Open Source, Jest Documentation. Meta Platforms Inc., 2024. E-BOOK.
Elérhető: https://jestjs.io/docs

[9] Jasmine Core Team, Jasmine Documentation. Jasmine, 2024. E-BOOK.
Elérhető: https://jasmine.github.io/

[10] Karma Team, Karma Test Runner Documentation. Open Source Community, 2024. E-BOOK.
Elérhető: https://karma-runner.github.io/

[11] GitHub Inc., GitHub Documentation. GitHub Inc., 2024. E-BOOK.
Elérhető: https://docs.github.com/

[12] Discord Inc., Discord Developer Documentation. Discord Inc., 2024. E-BOOK.
Elérhető: https://discord.com/developers/docs